

Scientific Collaboration Network Analyzer — Backend Implementation Document

1. Overview

This Technical Design Document describes the backend implementation for the Scientific Collaboration Network Analyzer, a research collaboration management platform enabling universities and research organizations to manage publications, researchers, institutions, projects, conferences, and collaborations.

The backend is built using **FastAPI** with **Python 3.x**, leveraging **SQLAlchemy ORM** for data access, **PostgreSQL** as the primary database, and **Redis** for caching and session management. The architecture diagram illustrates the complete system topology, showing how clients connect through the FastAPI Application (Uvicorn ASGI) gateway to the Business Service Modules, which interact with the Data Access Layer and ultimately the Data & Storage tier.

2. Goals

- Implement a secure, scalable REST API serving Desktop App (PyQt/Tkinter), Web Client (React/Flask), and API Consumers (Postman/Scripts)
- Develop eight core Business Service Modules: User Management, Researcher & Institution, Publication Repository, Collaboration, Conference Management, Citation & Reference, Analytics & Reports, and Notification Service
- Establish robust Middleware & Cross-Cutting concerns including Auth Service (JWT/OAuth2), RBAC/Access Control, Validation & Errors, File Upload Handler, and Audit Logging
- Integrate with External Services (CrossRef/DOI/ORCID, Email/SMTP Provider)
- Deploy using Docker containers with GitHub Actions CI/CD pipeline

3. Non-Goals

- AI-based analysis or machine learning features
 - Real-time collaborative editing capabilities
 - Mobile native application backend support
 - Multi-tenancy architecture (single-tenant deployment)
 - GraphQL API implementation
-

4. Architecture

The architecture diagram depicts a layered backend structure with the following tiers:

4.1 Client Layer

- **Desktop App** (PyQt/Tkinter)
- **Web Client** (React/Flask)
- **API Consumers** (Postman/Scripts) All clients communicate via HTTPS/REST to the gateway.

4.2 API Gateway Layer

- **FastAPI Application (Uvicorn ASGI)**
 - REST API routing
 - Pydantic validation
 - Request/response handling

4.3 Middleware & Cross-Cutting Layer

Component	Responsibility
Auth Service	JWT/OAuth2 token management
RBAC/Access Control	Role enforcement (Researcher, Inst. Admin, Admin)
Validation & Errors	Pydantic schema validation
File Upload Handler	Papers & metadata uploads
Audit Logging	Activity & security logs

4.4 Business Service Modules

Service	Features
User Management	Registration, Login, Roles, Profiles
Researcher & Institution	Profiles, Departments, Skills, Affiliations
Publication Repository	Journals, Conf. Papers, Books, Patents, Status
Collaboration	Co-authors, Projects, Teams, Partnerships
Conference Management	Registration, Presentations, Scheduling
Citation & Reference	Citations, DOI, Publication Linking
Analytics & Reports	Dashboards, Statistics, PDF/Excel Export
Notification Service	Email Alerts, System Messages

4.5 Data Access Layer

- **SQLAlchemy ORM** for database operations
- **Alembic Migrations** for schema versioning

4.6 Data & Storage Tier

- **PostgreSQL** — Primary relational database
- **Redis** — Cache & session storage
- **S3/Local Storage** — Paper file storage

4.7 External Services

- **CrossRef/DOI/ORCID** — Publication metadata and researcher identification
- **Email/SMTP Provider** — Notification delivery

4.8 Infrastructure & DevOps

- **Docker** containers
 - **GitHub Actions** CI/CD
 - **Python 3.x** runtime
 - **Monitoring** (Logs & Backups)
-

5. Data Model

5.1 Core Entities

User

```
users
├─ id: UUID (PK)
├─ email: VARCHAR(255) UNIQUE NOT NULL
├─ password_hash: VARCHAR(255) NOT NULL
├─ role: ENUM('researcher', 'institution_admin', 'reviewer', 'system_admin')
├─ is_active: BOOLEAN DEFAULT TRUE
├─ created_at: TIMESTAMP
└─ updated_at: TIMESTAMP
```

Researcher

```
researchers
├─ id: UUID (PK)
├─ user_id: UUID (FK → users.id)
├─ first_name: VARCHAR(100)
├─ last_name: VARCHAR(100)
├─ orcid: VARCHAR(19) UNIQUE
├─ department_id: UUID (FK → departments.id)
├─ bio: TEXT
├─ research_interests: JSONB
├─ skills: JSONB
└─ affiliations: JSONB
```

Institution

institutions

- └ id: UUID (PK)
- └ name: VARCHAR(255) NOT NULL
- └ type: ENUM('university', 'research_institute', 'government_lab', 'publisher', 'funding_org')
- └ country: VARCHAR(100)
- └ website: VARCHAR(255)
- └ created_at: TIMESTAMP

Department

departments

- └ id: UUID (PK)
- └ institution_id: UUID (FK → institutions.id)
- └ name: VARCHAR(255) NOT NULL
- └ description: TEXT

Publication

publications

- └ id: UUID (PK)
- └ title: VARCHAR(500) NOT NULL
- └ abstract: TEXT
- └ type: ENUM('journal', 'conference', 'book', 'patent', 'technical_report')
- └ status: ENUM('draft', 'submitted', 'published', 'archived')
- └ doi: VARCHAR(255) UNIQUE
- └ publication_date: DATE
- └ file_path: VARCHAR(500)
- └ metadata: JSONB
- └ created_at: TIMESTAMP
- └ updated_at: TIMESTAMP

PublicationAuthor (Junction Table)

publication_authors

- └ id: UUID (PK)
- └ publication_id: UUID (FK → publications.id)
- └ researcher_id: UUID (FK → researchers.id)
- └ author_order: INTEGER
- └ is_corresponding: BOOLEAN DEFAULT FALSE

Project

```
projects
├─ id: UUID (PK)
├─ title: VARCHAR(255) NOT NULL
├─ description: TEXT
├─ start_date: DATE
├─ end_date: DATE
├─ status: ENUM('active', 'completed', 'cancelled')
├─ funding_source: VARCHAR(255)
└─ created_at: TIMESTAMP
```

Conference

```
conferences
├─ id: UUID (PK)
├─ name: VARCHAR(255) NOT NULL
├─ location: VARCHAR(255)
├─ start_date: DATE
├─ end_date: DATE
├─ website: VARCHAR(255)
└─ created_at: TIMESTAMP
```

Citation

```
citations
├─ id: UUID (PK)
├─ citing_publication_id: UUID (FK → publications.id)
├─ cited_publication_id: UUID (FK → publications.id)
├─ citation_context: TEXT
└─ created_at: TIMESTAMP
```

AuditLog

```
audit_logs
├─ id: UUID (PK)
├─ user_id: UUID (FK → users.id)
├─ action: VARCHAR(100)
├─ entity_type: VARCHAR(100)
├─ entity_id: UUID
├─ details: JSONB
├─ ip_address: INET
└─ created_at: TIMESTAMP
```

5.2 Entity Relationships

- **User 1:1 Researcher**

- **Institution 1:N Department**
 - **Department 1:N Researcher**
 - **Publication N:M Researcher** (via publication_authors)
 - **Publication N:M Publication** (via citations - self-referential)
 - **Project N:M Researcher** (via project_members)
 - **Conference N:M Researcher** (via conference_participants)
-

6. API Design

6.1 API Conventions

- **Base URL:** /api/v1
- **Authentication:** Bearer token (JWT) in Authorization header
- **Content-Type:** application/json
- **Pagination:** ?page=1&limit=20
- **Sorting:** ?sort_by=created_at&order=desc

6.2 Authentication Endpoints

Method	Endpoint	Description
POST	/auth/register	User registration
POST	/auth/login	User login, returns JWT
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Invalidate session
GET	/auth/me	Get current user profile

6.3 User Management Endpoints

Method	Endpoint	Description	Roles
GET	/users	List all users	Admin
GET	/users/{id}	Get user by ID	Admin, Self
PUT	/users/{id}	Update user	Admin, Self
DELETE	/users/{id}	Deactivate user	Admin
PUT	/users/{id}/role	Update user role	Admin

6.4 Researcher Endpoints

Method	Endpoint	Description
GET	/researchers	List researchers (with filters)
GET	/researchers/{id}	Get researcher profile
POST	/researchers	Create researcher profile
PUT	/researchers/{id}	Update researcher profile
GET	/researchers/{id}/publications	Get researcher's publications
GET	/researchers/{id}/collaborators	Get co-authors
GET	/researchers/{id}/projects	Get associated projects

6.5 Publication Endpoints

Method	Endpoint	Description
GET	/publications	List publications (with filters)
GET	/publications/{id}	Get publication details
POST	/publications	Create publication
PUT	/publications/{id}	Update publication
DELETE	/publications/{id}	Delete publication
POST	/publications/{id}/upload	Upload publication file
PUT	/publications/{id}/status	Update publication status
GET	/publications/{id}/citations	Get citations
POST	/publications/{id}/authors	Add author

6.6 Collaboration Endpoints

Method	Endpoint	Description
GET	/projects	List projects
POST	/projects	Create project
PUT	/projects/{id}	Update project
POST	/projects/{id}/members	Add project member
GET	/collaborations	List institutional collaborations
POST	/collaborations	Create collaboration record

6.7 Conference Endpoints

Method	Endpoint	Description
GET	/conferences	List conferences
POST	/conferences	Create conference
PUT	/conferences/{id}	Update conference
POST	/conferences/{id}/register	Register for conference
POST	/conferences/{id}/presentations	Submit presentation

6.8 Citation & Reference Endpoints

Method	Endpoint	Description
POST	/citations	Create citation link
GET	/citations/publication/{id}	Get citations for publication
GET	/doi/lookup/{doi}	Lookup DOI via CrossRef

6.9 Analytics & Reports Endpoints

Method	Endpoint	Description
GET	/reports/publications	Publication statistics
GET	/reports/collaborations	Collaboration metrics
GET	/reports/institutions/{id}	Institution report
GET	/reports/researchers/{id}	Researcher analytics
GET	/export/pdf/{report_type}	Export report as PDF
GET	/export/excel/{report_type}	Export report as Excel

6.10 Response Format

Success Response

```
{
  "status": "success",
  "data": { },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 100
  }
}
```

Error Response

```
{
  "status": "error",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": []
  }
}
```

7. Security Considerations

7.1 Authentication & Authorization

- **JWT Tokens:** Access tokens (15-minute expiry), refresh tokens (7-day expiry)
- **Password Hashing:** bcrypt with salt rounds ≥ 12
- **OAuth2 Integration:** Support for institutional SSO
- **Session Management:** Redis-backed session storage with automatic expiration

7.2 Role-Based Access Control (RBAC)

Role	Permissions
Researcher	Manage own profile, publications, projects
Institution Admin	Manage institution data, view department analytics
Reviewer	Review publications, access review queue
System Admin	Full system access, user management, audit logs

7.3 Data Protection

- **HTTPS/TLS:** All communications encrypted (TLS 1.3)
- **Input Validation:** Pydantic schemas for all request bodies
- **SQL Injection Prevention:** SQLAlchemy parameterized queries
- **XSS Prevention:** Output encoding, Content-Security-Policy headers

- **CORS:** Whitelist-based origin validation

7.4 File Upload Security

- **File Type Validation:** Whitelist (PDF, DOC, DOCX)
- **Size Limits:** Maximum 50MB per file
- **Virus Scanning:** Integration point for antivirus scanning
- **Secure Storage:** S3 with server-side encryption or encrypted local storage

7.5 Audit Logging

All sensitive operations logged to the Audit Logging middleware:

- User authentication events
- Data modifications (create, update, delete)
- File uploads/downloads
- Permission changes
- Failed access attempts

7.6 Rate Limiting

- **Authentication endpoints:** 5 requests/minute per IP
 - **API endpoints:** 100 requests/minute per user
 - **File uploads:** 10 requests/hour per user
-

8. Testing Strategy

8.1 Unit Testing

- **Framework:** pytest
- **Coverage Target:** $\geq 80\%$
- **Scope:**
 - Pydantic schema validation
 - Service layer business logic
 - Utility functions
 - Data transformations

8.2 Integration Testing

- **Database Tests:** SQLAlchemy models with test PostgreSQL instance
- **API Tests:** FastAPI TestClient
- **External Service Mocks:** Responses library for CrossRef/ORCID API mocking

8.3 End-to-End Testing

- **Authentication Flows:** Registration → Login → Token refresh → Logout
- **Publication Workflow:** Create → Upload file → Add authors → Submit → Publish

- **Collaboration Workflow:** Create project → Add members → Track progress

8.4 Performance Testing

- **Tool:** Locust
- **Targets:**
 - API response time < 200ms (p95)
 - Support 100 concurrent users
 - Database query time < 50ms (p95)

8.5 Security Testing

- **Static Analysis:** Bandit for Python security linting
 - **Dependency Scanning:** Safety for vulnerability detection
 - **API Security:** OWASP ZAP for endpoint testing
-

9. Rollout Plan

9.1 Phase 1: Foundation (Milestone 1 — Weeks 1-2)

Deliverables:

- FastAPI project structure and configuration
- PostgreSQL database schema (Alembic migrations)
- Auth Service implementation (JWT/OAuth2)
- User Management service
- Researcher & Institution service (basic CRUD) **Validation Criteria:**
- Authentication endpoints functional
- User registration and login working
- Researcher profile creation operational
- Database migrations executing successfully

9.2 Phase 2: Core Modules (Milestone 2 — Weeks 3-4)

Deliverables:

- Publication Repository service
- File Upload Handler integration
- Conference Management service
- Institution management workflows

- Validation & Errors middleware refinement **Validation Criteria:**
- Publication CRUD with file upload working
- Publication status workflow (Draft → Submitted → Published → Archived)
- Conference registration functional
- Institution and department management operational

9.3 Phase 3: Advanced Features (Milestone 3 — Weeks 5-6)

Deliverables:

- Collaboration service (co-authors, projects, teams)
- Citation & Reference service
- Analytics & Reports service
- Notification Service (email integration)
- Dashboard data endpoints **Validation Criteria:**
- Collaboration tracking functional
- Citation linking with DOI lookup working
- Report generation (PDF/Excel export) operational
- Email notifications sending successfully

9.4 Phase 4: Production Readiness (Milestone 4 — Weeks 7-8)

Deliverables:

- Comprehensive test suite
- Performance optimization
- Docker containerization
- GitHub Actions CI/CD pipeline
- API documentation (OpenAPI/Swagger)
- Monitoring and logging setup **Validation Criteria:**
- All tests passing with $\geq 80\%$ coverage
- API response times meeting targets
- Docker deployment successful
- CI/CD pipeline executing on push

- Complete API documentation available

9.5 Deployment Architecture

```
Production Environment
├─ Docker Compose / Kubernetes
│   ├─ fastapi-app (2+ replicas)
│   ├─ postgresql (primary + replica)
│   ├─ redis (cluster mode)
│   └─ nginx (reverse proxy)
├─ S3 / MinIO (file storage)
└─ Monitoring Stack
    ├─ Prometheus (metrics)
    ├─ Grafana (dashboards)
    └─ ELK Stack (logs)
```

9.6 Rollback Strategy

- **Database:** Alembic downgrade migrations for schema rollback
- **Application:** Docker image versioning for instant rollback
- **Feature Flags:** Gradual feature enablement with kill switches
- **Backup:** Daily PostgreSQL backups with 30-day retention